

## ELE 216 Laboratoire 5 : Structures de données génériques

---

Le thème des laboratoires 5 et 6 est un banc de test automatisé pour la validation de matériel électronique. Au laboratoire 5, vous construirez les trois modules de base : un type résultat de test, un journal de résultats et un séquenceur de commandes. Au laboratoire 6, vous intégrerez ces modules dans une application interactive avec interpréteur de commandes et chargement de fichiers.

**Prérequis :** vous devez disposer des implémentations du Vecteur, de la Pile et de la File vues en cours. Placez-les dans un répertoire `../commun` et ajoutez `-I../commun` dans la variable `INCLUDES` de votre Makefile.

Pour chacun des problèmes, écrivez un module complet : un fichier d'en-tête (.h) qui définit l'interface publique, un fichier source (.c) qui implémente le module avec un type opaque, un fichier de tests unitaires utilisant Unity, et un Makefile. Documentez vos fichiers d'en-tête en format Doxygen.

Vous devez réaliser ce laboratoire seul, pendant la période. Il vous est permis de discuter avec vos collègues, consulter les chargés de laboratoires et l'internet.

Il est permis de réutiliser des fonctions réalisées dans les numéros précédents pour les numéros suivants.

**Vous devez faire valider votre travail avant de quitter.** On vous demandera d'expliquer clairement le fonctionnement de certains éléments de votre implémentation.

### Problème 1 : Résultat de test

Ce module encapsule un résultat de test individuel. Le verdict (PASS ou FAIL) est calculé automatiquement à la création et ne peut pas être modifié par la suite.

#### Verdict :

Le verdict est PASS si la valeur mesurée se trouve dans l'intervalle  $[\text{attendu} - \text{attendu} \times \text{tolérance} / 100, \text{attendu} + \text{attendu} \times \text{tolérance} / 100]$ . Sinon, le verdict est FAIL.

#### Interface :

`resultat_t *resultat_creer(const char *id, float attendu, float mesure, float tolerance);` alloue un résultat et calcule le verdict. Retourne NULL si id est NULL.

`void resultat_detruire(resultat_t *r);` libère la mémoire.

`const char *resultat_id(const resultat_t *r);` retourne l'identifiant du test.

`float resultat_attendu(const resultat_t *r) ;` retourne la valeur attendue.

`float resultat_mesure(const resultat_t *r) ;` retourne la valeur mesurée.

`float resultat_tolerance(const resultat_t *r) ;` retourne la tolérance en pourcentage.

`int resultat_est_reussi(const resultat_t *r) ;` retourne 1 si PASS, 0 si FAIL.

`void resultat_afficher(const resultat_t *r) ;` affiche tous les champs sur une ligne formatée.

### Tests unitaires :

- création valide, lecture correcte de tous les champs
- verdict PASS : mesure dans la tolérance
- verdict FAIL : mesure hors tolérance
- cas limite : tolérance de 0 % (seule la valeur exacte passe)
- cas limite : valeur attendue de 0 (la tolérance absolue est alors 0)
- création avec id NULL retourne NULL

## Problème 2 : Journal de résultats

Le journal accumule des résultats de test dans un **Vecteur** générique. On ne connaît pas à l'avance le nombre de tests ; le Vecteur est donc le conteneur naturel.

### Interface :

`journal_t *journal_creer(void) ;` alloue le journal et son Vecteur interne.

`void journal_detruire(journal_t *j) ;` libère tous les résultats stockés, le Vecteur interne et le journal.

`void journalajouter(journal_t *j, const char *id, float attendu, float mesure, float tolerance) ;` crée un `resultat_t` (P1), et l'ajoute au Vecteur.

`int journal_nb_resultats(const journal_t *j) ;` retourne le nombre total de résultats.

`int journal_nb_reussis(const journal_t *j) ;` retourne le nombre de résultats PASS.

`int journal_nb_echoues(const journal_t *j);` retourne le nombre de résultats FAIL.

`const resultat_t *journal_obtenir(const journal_t *j, int index);` retourne un pointeur const vers le résultat à l'index donné, ou NULL si l'index est invalide.

`void journal_afficher(const journal_t *j);` affiche un rapport tabulaire de tous les résultats, suivi d'un résumé (total, réussis, échoués).

`void journal_sauvegarder(const journal_t *j, const char *chemin);` écrit le rapport dans un fichier texte.

### Tests unitaires :

- création et destruction sans fuite
- ajout de résultats, compteurs cohérents (total, réussis, échoués)
- obtenir un résultat par index valide : champs corrects
- obtenir par index invalide retourne NULL
- mélange de PASS et FAIL, compteurs corrects

## Problème 3 : Séquenceur de commandes

Le séquenceur gère une **File** de commandes à exécuter et une **Pile** de contextes de configuration. Avant d'exécuter un groupe de tests sous une configuration différente, on empile (push) le contexte courant ; après, on le dépile (pop) pour revenir à l'état précédent.

### Contexte de configuration :

Définissez une structure `config_t` contenant :

- une tension d'alimentation (float, en volts)
- un courant limite (float, en ampères)
- une tolérance par défaut (float, en pourcentage)

### Commande :

Définissez une structure `commande_t` contenant :

- un type (enum : `CMD_SET_TENSION`, `CMD_SET_COURANT`, `CMD_SET_TOLERANCE`, `CMD_MESURER`, `CMD_PUSH`, `CMD_POP`, `CMD_RAPPORT`)
- un identifiant de test (`char[]`)
- une valeur numérique (float)

## Interface :

`sequenceur_t *sequenceur_creer(void)` ; alloue le séquenceur, sa File de commandes, sa Pile de contextes, et initialise un contexte par défaut (tension = 0, courant = 0, tolérance = 5 %).

`void sequenceur_detruire(sequenceur_t *s)` ; libère le séquenceur et toutes ses structures internes.

`void sequenceur_enfiler(sequenceur_t *s, commande_t cmd)` ; ajoute une commande à la file d'attente.

`int sequenceur_nb_en_attente(const sequenceur_t *s)` ; retourne le nombre de commandes en attente.

`int sequenceur_executer_prochaine(sequenceur_t *s, journal_t *j)` ; défile et exécute une commande. Retourne 1 si une commande a été exécutée, 0 si la file était vide.

`void sequenceur_executer_tout(sequenceur_t *s, journal_t *j)` ; exécute toutes les commandes en attente.

`const config_t *sequenceur_contexte(const sequenceur_t *s)` ; retourne un pointeur const vers le contexte de configuration courant.

`void sequenceur_afficher_file(const sequenceur_t *s)` ; affiche les commandes en attente.

## Comportement de chaque type de commande :

`CMD_SET_TENSION`, `CMD_SET_COURANT`, `CMD_SET_TOLERANCE` — modifie le champ correspondant du contexte courant.

`CMD_PUSH` — empile une copie du contexte courant.

`CMD_POP` — dépile le contexte. Si la pile est vide, la commande est ignorée avec un message d'avertissement.

`CMD_MESURER` — simule une mesure en générant une valeur aléatoire perturbée autour de la valeur nominale (valeur  $\pm 10$  % de bruit). Ajoute le résultat au journal avec la tolérance du contexte courant.

`CMD_RAPPORT` — appelle `journal_afficher()` sur le journal.

### **Tests unitaires :**

- création et destruction sans fuite
- enfilage de commandes et compteur en attente
- exécution d'une commande `CMD_SET_TENSION` : le contexte est modifié correctement
- `CMD_PUSH` puis modification puis `CMD_POP` : le contexte est restauré
- `CMD_POP` sur une pile vide : pas de plantage
- `CMD_MESURER` : un résultat est ajouté au journal (fixer le seed aléatoire avec `srand()` pour des tests reproductibles)
- exécution depuis une file vide retourne 0